ЧАСТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«СУРГУТСКИЙ КОЛЛЕДЖ ПРЕДПРИНИМАТЕЛЬСТВА» (ЧПОУ «СКП»)

КУРСОВОЙ ПРОЕКТ

Тема: «Проектирование и разработка модуля учета движения канцелярских товаров»

ПО ПМ.01 РАЗРАБОТКА МОДУЛЕЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ДЛЯ КОМПЬЮТЕРНЫХ СИСТЕМ

МДК.01.01 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

Специальность 09.02.07 «Информационные системы и программирование»

Выполнил:
студент группы ИСиП-23/1
[Фамилия Имя Отчество]

Руководитель:
Трушина Александра Игоревна

Сургут, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....5
1.1 Учёт движения канцелярских товаров в организации.....5
1.2 Обзор аналогов.....6
1.3 Требования к модулю.....6
1.4 Выбор средств разработки.....8
Выводы по разделу 1.....8
2 ПРОЕКТИРОВАНИЕ МОДУЛЯ.....10
2.1 Модель данных.....10
2.2 Главное правило и расчёт остатка.....12
2.3 Архитектура модуля.....12
2.4 Алгоритм оформления выдачи.....13
Выводы по разделу 2.....14
3 РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ МОДУЛЯ.....16
3.1 Справочник товаров.....16
3.2 Поступление и выдача.....17
3.3 Расчёт остатка и контроль минимума.....19
3.4 История движения и итоги за период.....20
3.5 Хранение данных и развёртывание.....22
3.6 Тестирование программного модуля.....22
Выводы по разделу 3.....25
ЗАКЛЮЧЕНИЕ.....26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....27
ПРИЛОЖЕНИЕ А. ЛИСТИНГ ТЕСТОВ РАСЧЁТА ОСТАТКА.....29
ПРИЛОЖЕНИЕ Б. ЛИСТИНГ КЛАССА РЕЗУЛЬТАТА ОПЕРАЦИИ.....30

ВВЕДЕНИЕ

В любой организации ежедневно расходуются канцелярские товары: бумага, ручки, папки, скрепки, картриджи. Их закупают партиями у поставщиков и выдают сотрудникам по мере необходимости. Если учёт ведётся в тетради или в разрозненных таблицах, кладовщик не может быстро ответить на простые вопросы: сколько бумаги осталось на складе, кто и когда получил степлеры, какие товары пора заказывать. Ошибки в ручном пересчёте приводят к тому, что товар заканчивается неожиданно, а сотрудники вынуждены ждать новую поставку.

Актуальность темы определяется тем, что учёт материальных запасов – обязательная часть хозяйственной деятельности организации, а для небольших подразделений полноценные учётные системы избыточны и дороги. Требуется простой модуль, который хранит историю движения товаров, сам считает остаток и предупреждает о необходимости пополнения.

Степень разработанности проблемы. Задачи складского учёта решаются как крупными учётными системами (например, «1С:Бухгалтерия», облачный сервис «МойСклад»), так и электронными таблицами. Первые требуют лицензий, настройки и обучения, вторые не защищают от ошибок ввода: в таблице можно случайно выдать больше товара, чем есть, или изменить уже внесённую запись. Поэтому разработка небольшого специализированного модуля с жёсткими правилами проверки данных остаётся практически значимой.

Объект исследования – процесс учёта поступления и выдачи канцелярских товаров в организации.

Предмет исследования – программный модуль учёта движения канцелярских товаров с автоматическим расчётом остатка и контролем минимального количества.

Цель работы – спроектировать и разработать модуль учёта движения канцелярских товаров на платформе .NET 10, который ведёт справочник товаров, регистрирует операции поступления и выдачи, автоматически рассчитывает

текущий остаток, хранит историю движения и контролирует минимальное количество товара.

Для достижения цели поставлены задачи:

– проанализировать предметную область и сформулировать требования к модулю;

– выбрать и обосновать средства разработки;

– спроектировать модель данных, архитектуру и алгоритм главной операции;

– реализовать справочник товаров, операции поступления и выдачи, расчёт остатка, историю движения и контроль минимума;

– разработать автоматические тесты и выполнить ручные проверки всех обязательных функций;

– подготовить инструкцию запуска и инструкцию пользователя.

Методы исследования: анализ предметной области и аналогов, объектно-ориентированное проектирование, моделирование данных (ER-диаграмма), модульное тестирование, ручное функциональное тестирование.

Теоретическая значимость работы состоит в систематизации правил учёта движения товаров в виде формализованных ограничений и формулы расчёта остатка. Практическая значимость – модуль готов к использованию в небольшой организации или учебном заведении: он запускается одной командой, хранит данные в файле базы SQLite и не требует отдельного сервера базы данных.

Работа состоит из введения, трёх разделов, заключения, списка использованных источников и приложений. В первом разделе проанализирована предметная область и сформулированы требования, во втором описано проектирование модуля, в третьем – реализация, развёртывание и тестирование.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Учёт движения канцелярских товаров в организации

Канцелярские товары относятся к материально-производственным запасам организации. Порядок их учёта определяется федеральным законом «О бухгалтерском учёте» [1] и федеральным стандартом ФСБУ 5/2019 «Запасы» [2]: каждое поступление и каждое списание должно подтверждаться документом, а остаток на любую дату должен быть восстановим по первичным записям.

На практике движение канцтоваров выглядит так. Офис-менеджер или кладовщик получает товар от поставщика по товарной накладной или универсальному передаточному документу (далее – УПД) и фиксирует поступление. Затем по заявкам выдаёт товар сотрудникам разных отделов и фиксирует, кто, что и в каком количестве получил. Периодически ответственный проверяет, каких позиций осталось мало, и формирует заказ поставщику.

Без автоматизации в этом процессе возникают типичные проблемы:

– остаток пересчитывается вручную и легко расходится с фактическим количеством;

– можно записать выдачу большего количества, чем есть на складе, и обнаружить ошибку только при инвентаризации;

– нет единой истории: чтобы узнать, кто получил товар месяц назад, приходится перелистывать журнал;

– о том, что товар заканчивается, узнают, когда он уже закончился;

– исправление ошибки «зачёркиванием» уничтожает историю, и остаток становится невозможно проверить.

Из этого следует главное требование к модулю: остаток не должен вводиться руками, он должен вычисляться по истории операций, а операции, нарушающие правила, должны отклоняться до сохранения.

1.2 Обзор аналогов

Рассмотрены три распространённых способа учёта: учётная система «1С:Бухгалтерия», облачный сервис «МойСклад» и электронная таблица Microsoft Excel. Результаты сравнения приведены в таблице 1.

Таблица 1 – Сравнение аналогов

Критерий	**«1С:Бухгалтерия»**	**«МойСклад»**	**Таблица Excel**	**Разрабатываемый модуль**
Автоматический расчёт остатка	да	да	только формулами, вручную	да
Запрет выдачи сверх остатка	настраивается	да	нет	да
Контроль минимального остатка	да	да	нет	да
История движения	да	да	частично	да
Стоимость	платная лицензия	подписка	входит в офисный пакет	бесплатно
Сложность освоения	высокая	средняя	низкая	низкая
Работа без интернета и сервера	да (локальная установка)	нет	да	да

Полнофункциональные системы решают задачу, но избыточны для учёта одной группы товаров в небольшом подразделении. Электронная таблица проста, но не защищает данные от ошибок. Разрабатываемый модуль занимает промежуточное положение: он содержит только нужные функции, но строго проверяет каждую операцию.

1.3 Требования к модулю

Функциональные требования сформулированы на основе задания: «Реализовать справочник товаров, операции поступления и выдачи, автоматический расчёт текущего остатка и историю движения. Добавить контроль минимального количества товара». Перечень функций приведён в таблице 2.

Таблица 2 – Функциональные требования

Код	Функция	Содержание
Ф1	Справочник товаров	добавление, просмотр, изменение, архивирование; поиск по наименованию и артикулу
Ф2	Поступление	количество, дата, поставщик или номер документа, комментарий
Ф3	Выдача	количество, дата, сотрудник-получатель, комментарий; запрет выдачи сверх остатка
Ф4	Расчёт остатка	остаток вычисляется по операциям и не вводится вручную
Ф5	История движения	список операций с фильтрами по товару, типу, сотруднику, периоду; история в карточке товара
Ф6	Контроль минимума	пометка «Требует пополнения», фильтр «Ниже минимума», предупреждение при выдаче
Ф7	Итоги за период	сколько поступило и сколько выдано по каждому товару
Ф8	Справочник сотрудников	добавление и изменение получателей

Ограничения предметной области:

– количество в операции – целое число больше нуля;

– для выдачи обязателен сотрудник-получатель, для поступления – поставщик или номер документа;

– дата операции не может быть в будущем;

– с архивным товаром операции не проводятся;

– артикул товара уникален, минимальный остаток не меньше нуля;

– операции не редактируются и не удаляются: ошибка исправляется новой корректирующей операцией с комментарием.

Нефункциональные требования:

– платформа .NET 10, язык C#, веб-приложение ASP.NET Core MVC;

– хранение данных в файле базы SQLite, база создаётся автоматически при запуске;

– все надписи и сообщения об ошибках на русском языке;

– неверные данные и обращение к несуществующей записи не приводят к аварийному завершению;

– предметная логика отделена от контроллеров и проверяется автоматическими тестами;

– исходный код хранится в системе контроля версий Git.

1.4 Выбор средств разработки

Средства разработки выбраны с учётом изученного на занятиях материала и требований к модулю (таблица 3).

Таблица 3 – Средства разработки

Средство	Версия	Назначение и причина выбора
C#	14	основной язык: строгая типизация, LINQ для запросов к данным [5]
.NET SDK	10.0	платформа с долгосрочной поддержкой (LTS), кроссплатформенная
ASP.NET Core MVC	10.0	веб-приложение по шаблону «модель – представление – контроллер» [4], [9]
Entity Framework Core	10.0.15	объектно-реляционное отображение, миграции базы [6], [10]
SQLite	3 (через Microsoft.EntityFrameworkCore.Sqlite)	встраиваемая база в одном файле, не требует сервера
Bootstrap	5.3.3	оформление страниц без собственного CSS-фреймворка
xUnit	2.9.3	автоматические тесты
Git, GitHub	2.x	контроль версий, история этапов, автоматическая сборка и тесты (GitHub Actions) [8]

Выбор ASP.NET Core MVC вместо Web API обусловлен тем, что пользователю модуля – кладовщику – нужны готовые страницы с формами и таблицами, а не программный интерфейс. При этом для ручных проверок HTTP-запросами в модуль добавлены несколько API-методов, возвращающих данные в формате JSON.

SQLite выбрана потому, что модуль рассчитан на одного пользователя или небольшой отдел: база хранится в одном файле, её легко скопировать и восстановить, а EF Core позволяет при необходимости перейти на другую СУБД без изменения предметной логики.

Выводы по разделу 1

В разделе проанализирован процесс учёта канцелярских товаров и выявлены проблемы ручного учёта: расхождение остатка, выдача сверх наличия, отсутствие истории и несвоевременное пополнение. Сравнение аналогов показало, что для небольшого подразделения нужен простой модуль со строгими

проверками. Сформулированы восемь функциональных требований, ограничения предметной области и нефункциональные требования, выбраны средства разработки: C#, .NET 10, ASP.NET Core MVC, EF Core, SQLite, xUnit и Git.

2 ПРОЕКТИРОВАНИЕ МОДУЛЯ

2.1 Модель данных

Модель данных состоит из трёх сущностей: товар (Product), сотрудник (Employee) и операция движения (StockOperation). Товар и сотрудник – справочники, операция – журнал, в который добавляются записи о поступлениях и выдачах. Связи и поля показаны на рисунке 1 в соответствии с ER-диаграммой.

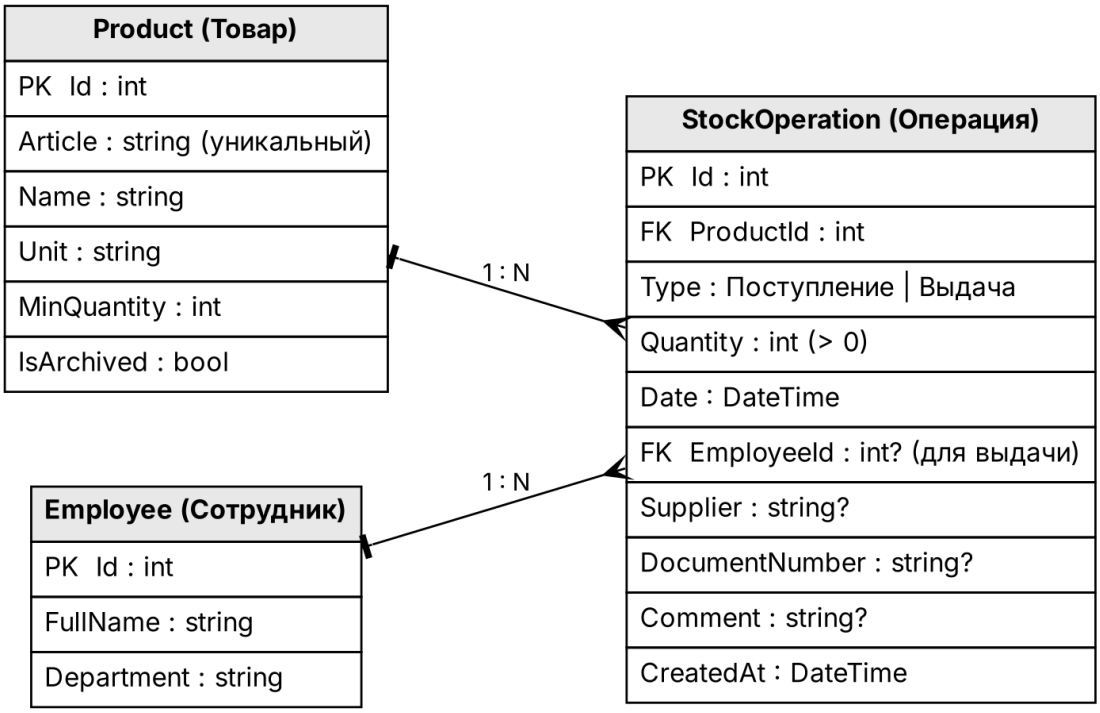


Рисунок 1 – ER-диаграмма модели данных

Описание сущностей приведено в таблице 4.

Таблица 4 – Сущности модели данных

Сущность	Поля	Ограничения
Product (товар)	Id, Article, Name, Unit, MinQuantity, IsArchived	артикул обязателен и уникален (уникальный индекс); MinQuantity ≥ 0; товар не удаляется, а архивируется
Employee (сотрудник)	Id, FullName, Department	ФИО и отдел обязательны; сотрудник не удаляется, так как на него ссылаются выдачи
StockOperation (операция)	Id, ProductId, Type, Quantity, Date, EmployeeId, Supplier, DocumentNumber, Comment, CreatedAt	Quantity > 0; EmployeeId обязателен для выдачи; Supplier или DocumentNumber обязателен для поступления; запись не изменяется

Связь «товар – операции» имеет тип «один ко многим»: у товара может быть сколько угодно операций, каждая операция относится ровно к одному товару. Связь «сотрудник – операции» также «один ко многим», но необязательная: у поступлений сотрудника нет. Для обеих связей задано правило удаления Restrict – база не позволит удалить товар или сотрудника, у которых есть операции. Это защищает историю движения.

Сущности описаны классами C#, а ограничения ввода – атрибутами проверки данных. Класс товара приведён в листинге 1: атрибут [Required] делает поле обязательным, [StringLength] ограничивает длину, [Range] задаёт допустимый диапазон, а [Display] – русскую подпись поля на странице.

Листинг 1 – Класс сущности «Товар» (Models/Product.cs, сокращённо)

```
public class Product
{
    public int Id { get; set; }

    [Display(Name = "Артикул")]
    [Required(ErrorMessage = "Укажите артикул")]
    [StringLength(20, ErrorMessage = "Артикул — не длиннее 20 символов")]
    public string Article { get; set; } = string.Empty;

    [Display(Name = "Наименование")]
    [Required(ErrorMessage = "Укажите наименование")]
    public string Name { get; set; } = string.Empty;

    [Display(Name = "Минимальный остаток")]
    [Range(0, 100000, ErrorMessage = "Минимальный остаток должен быть от 0 до
100000")]
    public int MinQuantity { get; set; }

    public bool IsArchived { get; set; }          // архив вместо удаления
    public List<StockOperation> Operations { get; set; } = new();   // связь
1 — N
}
```

Связи «один ко многим», правило Restrict и уникальный индекс по артикулу настраиваются в методе OnModelCreating контекста AppDbContext; по этой настройке создаётся миграция.

Тип операции хранится перечислением OperationType со значениями Receipt (поступление) и Issue (выдача). В таблице операций нет поля «остаток» – это принципиальное проектное решение, следующее из главного правила.

2.2 Главное правило и расчёт остатка

Главное правило модуля: нельзя выдать больше товара, чем есть на складе. Остаток при этом не хранится, а вычисляется по формуле (1):

$$O = \Sigma Q_{пост} - \Sigma Q_{выд} \quad (1)$$

где O – текущий остаток товара;

$\Sigma Q_{пост}$ – сумма количеств во всех операциях поступления данного товара;

$\Sigma Q_{выд}$ – сумма количеств во всех операциях выдачи данного товара.

Выдача количества q разрешается, если выполняется условие (2):

$$q \le O \quad (2)$$

где q – запрошенное количество для выдачи.

Признак «Требует пополнения» устанавливается по условию (3):

$$O \le O_{min} \quad (3)$$

где O_{min} – минимальный остаток товара (поле MinQuantity).

Из формул следуют граничные случаи, которые проверяются тестами: выдача ровно всего остатка допустима (остаток становится 0); выдача «остаток + 1» отклоняется; остаток, равный минимуму, уже означает «Требует пополнения»; у нового товара без операций остаток равен нулю. Если после выдачи остаток опускается до минимума или ниже, выдача выполняется, но пользователь получает предупреждение.

2.3 Архитектура модуля

Модуль построен по многослойной архитектуре (рисунок 2). Каждый слой выполняет свою задачу и обращается только к нижележащему.

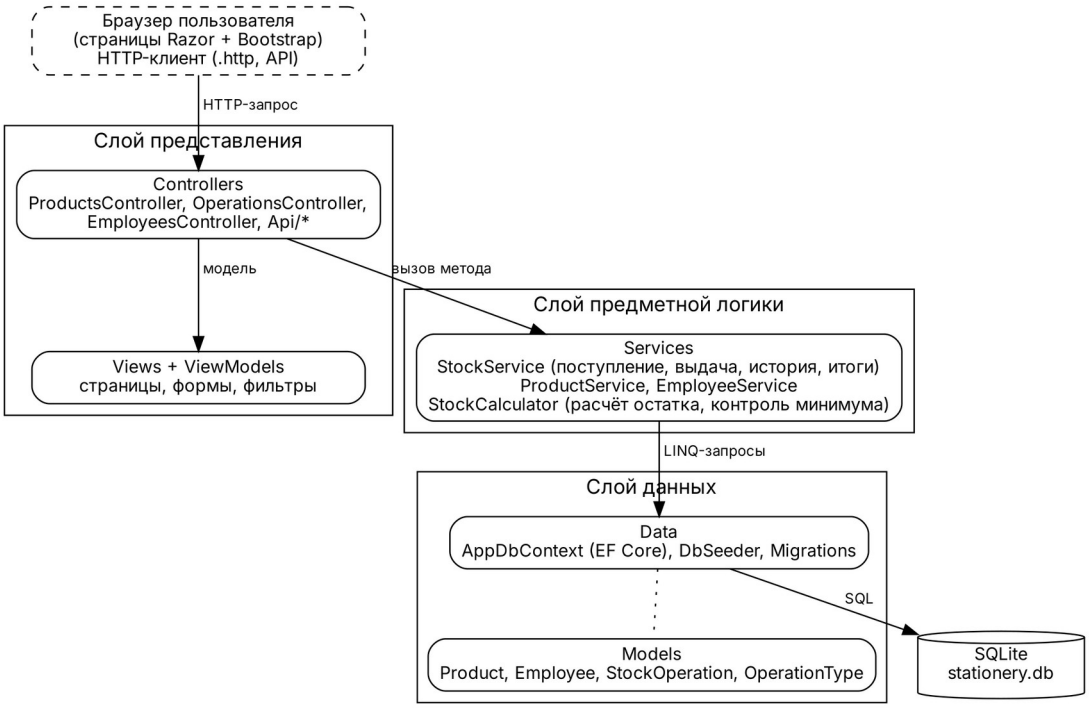


Рисунок 2 – Архитектура модуля

Слой представления – контроллеры и представления Razor. Контроллер принимает HTTP-запрос, вызывает метод сервиса и выбирает страницу для ответа; в нём нет ни расчётов, ни обращений к базе. Слой предметной логики – классы папки Services: StockService (поступление, выдача, история, итоги), ProductService, EmployeeService и статический класс StockCalculator с чистыми функциями расчёта. Слой данных – контекст AppDbContext, миграции и начальные учебные данные (DbSeeder).

Сервисы возвращают результат в виде объекта OperationResult со свойствами Success, Error, Warning и IsNotFound. Ошибка пользователя – это не исключение, а обычный результат с понятным текстом на русском языке, который контроллер показывает на странице. Папки проекта: Models (сущности), Data (контекст, учебные данные, миграции), Services (логика), ViewModels (формы и фильтры), Controllers и Views (страницы), StationeryAccounting.Tests (тесты).

2.4 Алгоритм оформления выдачи

Выдача – главная операция модуля. Порядок проверок показан на блок-схеме в соответствии с рисунком 3 (оформлена по ГОСТ 19.701-90 [3]). Сначала

проверяются данные, которые не требуют обращения к базе (количество и дата), затем наличие получателя, существование и состояние товара, и только после этого рассчитывается остаток. Такой порядок позволяет как можно раньше отклонить неверный запрос.

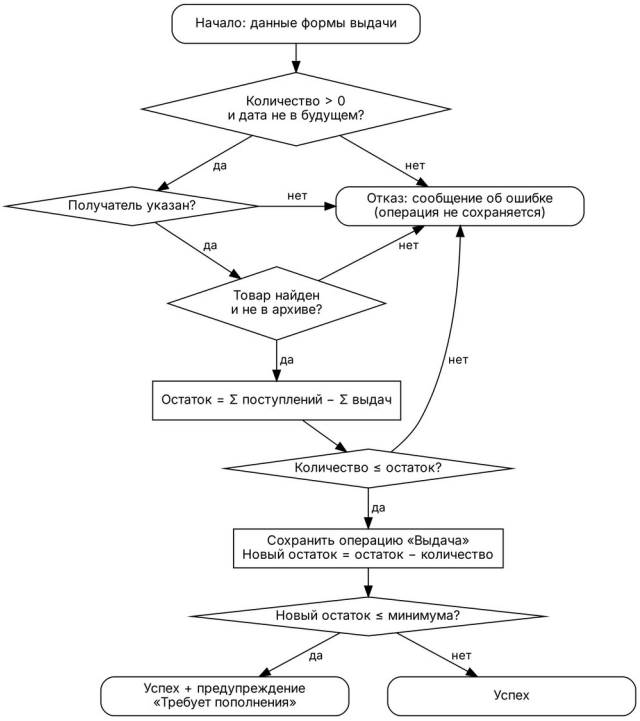


Рисунок 3 – Блок-схема алгоритма оформления выдачи

Проверка остатка и сохранение выдачи выполняются внутри критической секции (оператор lock). Если два пользователя одновременно оформят выдачу одного товара, вторая операция начнёт проверку только после сохранения первой и увидит уже уменьшенный остаток. Без этого обе операции могли бы пройти проверку и вместе выдать больше, чем есть на складе.

Выводы по разделу 2

Спроектирована модель данных из трёх сущностей со связями «один ко многим»; остаток не хранится, а вычисляется по формуле (1), что исключает его расхождение с историей. Сформулированы условия выдачи и пополнения в виде формул (2) и (3) и выделены граничные случаи. Выбрана трёхслойная архитектура с отделением предметной логики от контроллеров, построены

диаграмма вариантов использования и блок-схема главной операции с защитой от одновременных выдач.

3 РЕАЛИЗАЦИЯ И ТЕСТИРОВАНИЕ МОДУЛЯ

3.1 Справочник товаров

Справочник товаров реализован контроллером ProductsController и сервисом ProductService. На главной странице справочника, представленной на рисунке 4, для каждого товара выводятся артикул, наименование, единица измерения, текущий остаток и минимальный остаток. Строки товаров, требующих пополнения, выделены красным цветом и помечены надписью «Требует пополнения».

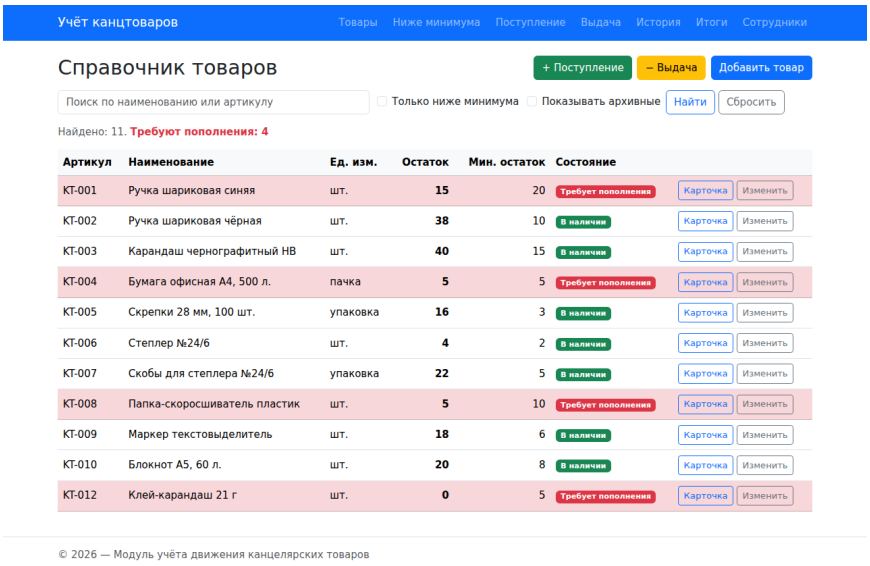


Рисунок 4 – Справочник товаров с текущим остатком

Остатки для списка рассчитываются одним запросом: операции группируются по товару, и для каждой группы вызывается функция StockCalculator.CalculateBalance. Поиск выполняется по части наименования или артикула без учёта регистра (листинг 2):

Листинг 2 – Поиск в справочнике товаров (Services/ProductService.cs)

```
if (!string.IsNullOrWhiteSpace(filter.Search))
{
    string text = filter.Search.Trim();
    items = items.Where(i =>
        i.Name.Contains(text, StringComparison.OrdinalIgnoreCase) ||
        i.Article.Contains(text, StringComparison.OrdinalIgnoreCase));
}
```

Поиск выполняется в памяти, а не средствами базы: в SQLite сравнение без учёта регистра работает только для латинских букв, а наименования товаров

записаны по-русски. Для справочника из десятков и сотен позиций это не влияет на скорость.

Пример. Пользователь вводит в строку поиска «ручка». Метод Contains с параметром OrdinalIgnoreCase сравнивает строки без учёта регистра, поэтому находятся «Ручка шариковая синяя» и «Ручка шариковая чёрная», а запрос «kt-003» находит «Карандаш чернографитный НВ» по артикулу КТ-003.

При добавлении и изменении товара сервис убирает лишние пробелы, приводит артикул к верхнему регистру и проверяет его уникальность среди других товаров. Попытка добавить товар с артикулом «kt-001», который уже существует как «КТ-001», отклоняется с сообщением «Товар с артикулом „КТ-001“ уже существует».

Товар не удаляется: вместо удаления используется архивирование. Архивный товар скрыт в справочнике по умолчанию, не предлагается в формах операций, а попытка провести операцию с ним через API отклоняется. Так сохраняется история движения, по которой рассчитывается остаток.

3.2 Поступление и выдача

Операции реализованы в классе StockService. Фрагмент метода выдачи, содержащий главное правило, приведён в листинге 3.

Листинг 3 – Проверка остатка при выдаче (Services/StockService.cs, метод Issue)

```
lock (IssueLock)
{
    // ГЛАВНОЕ ПРАВИЛО: нельзя выдать больше, чем есть на складе
    int balance = GetBalance(product.Id);
    if (form.Quantity > balance)
    {
        string unit = Units.Short(product.Unit);
        return OperationResult.Fail(
            $"Недостаточно товара «{product.Name}»: на складе {balance}
{unit}, " +
            $"запрошено {form.Quantity} {unit}");
    }
    _db.StockOperations.Add(new StockOperation { ... });
    _db.SaveChanges();

    int newBalance = balance - form.Quantity;
    string? warning = null;
    if (StockCalculator.NeedsRestock(newBalance, product.MinQuantity))
        warning = $"Внимание: остаток «{product.Name}» — {newBalance} ...
Товар требует пополнения.";
    return OperationResult.Ok(warning);
}
```

Пример. На складе 38 шт. товара КТ-002 «Ручка шариковая чёрная». Пользователь оформляет выдачу 100 шт. Метод GetBalance возвращает 38, условие 100 > 38 истинно, поэтому метод возвращает отказ, операция не сохраняется, а на форме выдачи показывается сообщение «Недостаточно товара „Ручка шариковая чёрная“: на складе 38 шт., запрошено 100 шт.» (рисунок 5).

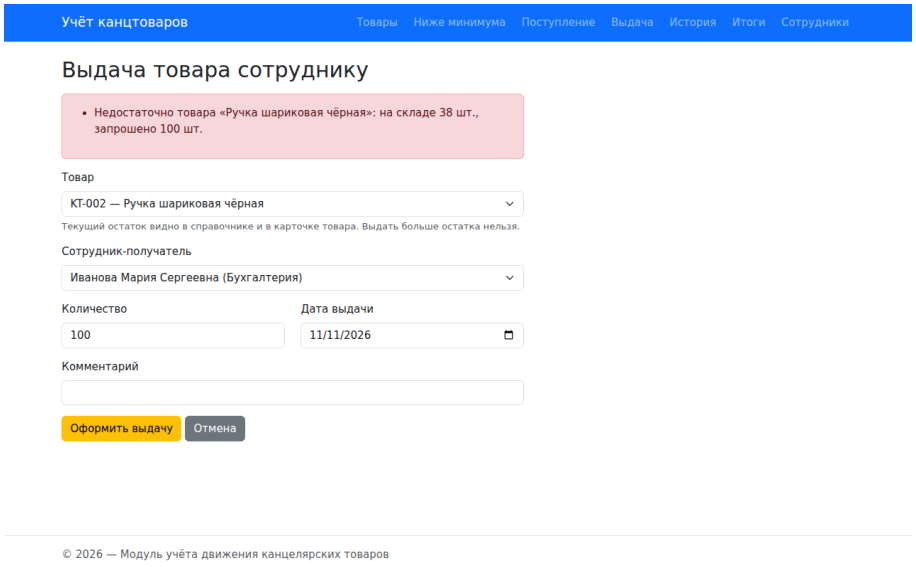


Рисунок 5 – Отклонение выдачи сверх остатка

Пример выдачи с предупреждением. Остаток скрепок КТ-005 – 16 уп., минимальный остаток – 3 уп. Выдача 13 уп. допустима (13 ≤ 16), новый остаток

16 − 13 = 3, и так как 3 ≤ 3, пользователь видит предупреждение о необходимости пополнения (рисунок 6).

Контроллер только передаёт форму в сервис и выбирает, какую страницу показать: при успехе – переход в карточку товара, при ошибке – та же форма с сообщением. Проверок и обращений к базе в нём нет – это требование разделения слоёв.

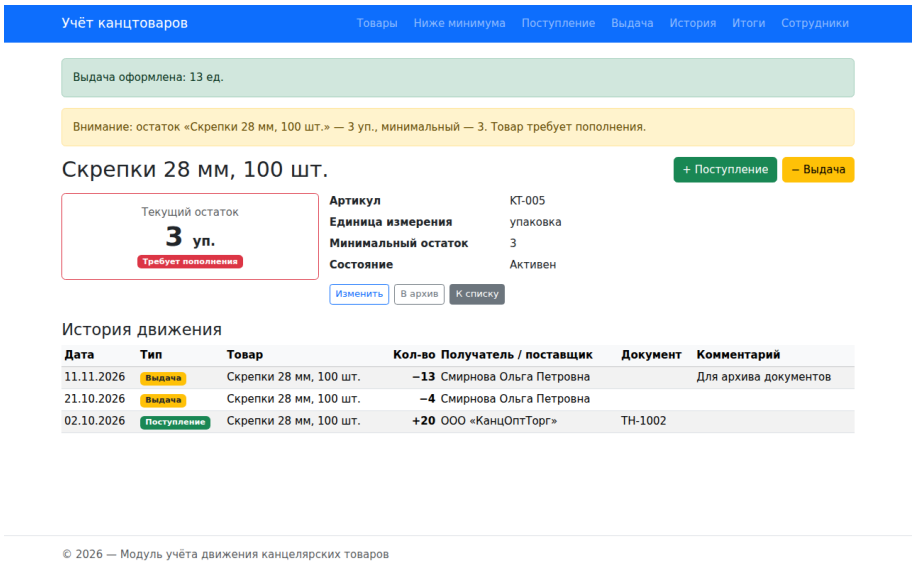


Рисунок 6 – Выдача с предупреждением о необходимости пополнения

Для поступления сервис проверяет, что указан поставщик или номер документа; пустые строки и строки из пробелов не считаются заполненными. Общие проверки количества и даты вынесены в отдельный метод CheckQuantityAndDate, который используют обе операции.

3.3 Расчёт остатка и контроль минимума

Формулы (1) и (3) реализованы в статическом классе StockCalculator (листинг 4).

Листинг 4 – Расчёт остатка и контроль минимума (Services/StockCalculator.cs)

```
public static int CalculateBalance(IEnumerable<StockOperation> operations)
{
    int received = operations.Where(o => o.Type ==
OperationType.Receipt).Sum(o => o.Quantity);
    int issued   = operations.Where(o => o.Type == OperationType.Issue).Sum(o
=> o.Quantity);
    return received - issued;
}

public static bool NeedsRestock(int balance, int minQuantity) => balance <=
minQuantity;
```

Пример расчёта для ручки КТ-001 по учебным данным: поступило 100 шт., выдано 30 + 25 + 30 = 85 шт., остаток О = 100 − 85 = 15 шт. Минимальный остаток – 20 шт., 15 ≤ 20, поэтому товар отмечен «Требует пополнения». Класс не обращается к базе данных, поэтому его функции проверяются тестами без подготовки базы. Остаток и состояние товара показаны в карточке товара вместе с историей движения именно этого товара. Отдельная страница «Ниже минимума» показывает только активные товары, требующие пополнения.

3.4 История движения и итоги за период

История движения реализована методом StockService.GetHistory (листинг 5). Условия фильтра добавляются к запросу по одному: запрос LINQ формируется как IQueryable и выполняется в базе только при вызове ToList(), поэтому из базы читаются только подходящие записи.

Листинг 5 – Фильтрация истории движения (Services/StockService.cs, метод GetHistory)

```
IQueryable<StockOperation> query = _db.StockOperations
    .AsNoTracking()
    .Include(o => o.Product)
    .Include(o => o.Employee);

if (filter.ProductId != null)  query = query.Where(o => o.ProductId ==
filter.ProductId);
if (filter.Type != null)       query = query.Where(o => o.Type ==
filter.Type);
if (filter.EmployeeId != null) query = query.Where(o => o.EmployeeId ==
filter.EmployeeId);
if (filter.From != null)       query = query.Where(o => o.Date >=
filter.From.Value.Date);
if (filter.To != null)         query = query.Where(o => o.Date <=
filter.To.Value.Date);

return query.OrderByDescending(o => o.Date).ThenByDescending(o =>
o.Id).ToList();
```

Пример: при фильтре «товар КТ-001, тип „Выдача“» в запрос попадают два условия Where, и база возвращает три выдачи ручки на сумму 85 шт. Пользователь может отобрать операции по товару, типу, сотруднику и периоду; над таблицей выводятся суммы поступлений и выдач по найденным операциям (рисунок 7).

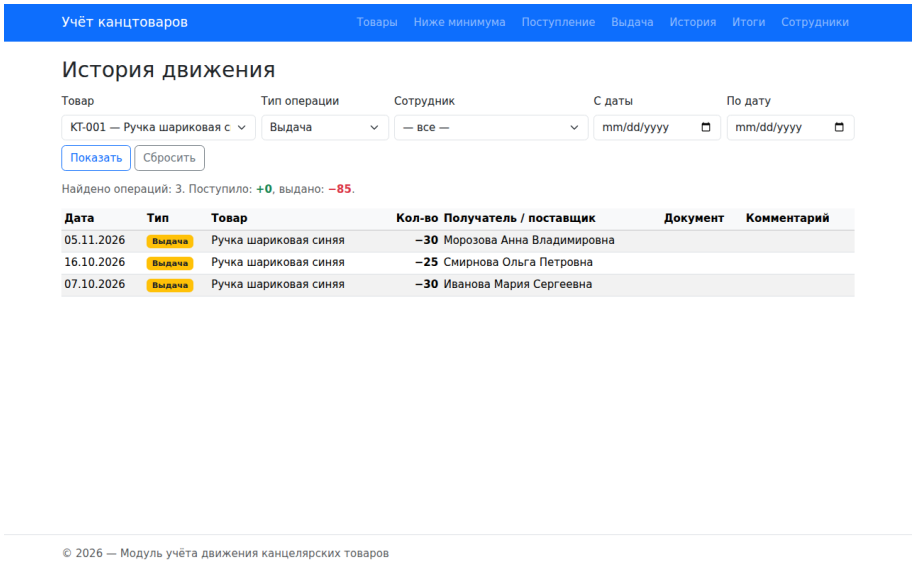


Рисунок 7 – История движения с фильтром по товару и типу операции

Итоги за период (рисунок 8) группируют операции по товару и показывают, сколько каждого товара поступило и выдано. Если дата начала периода позже даты окончания, вместо таблицы выводится сообщение об ошибке.

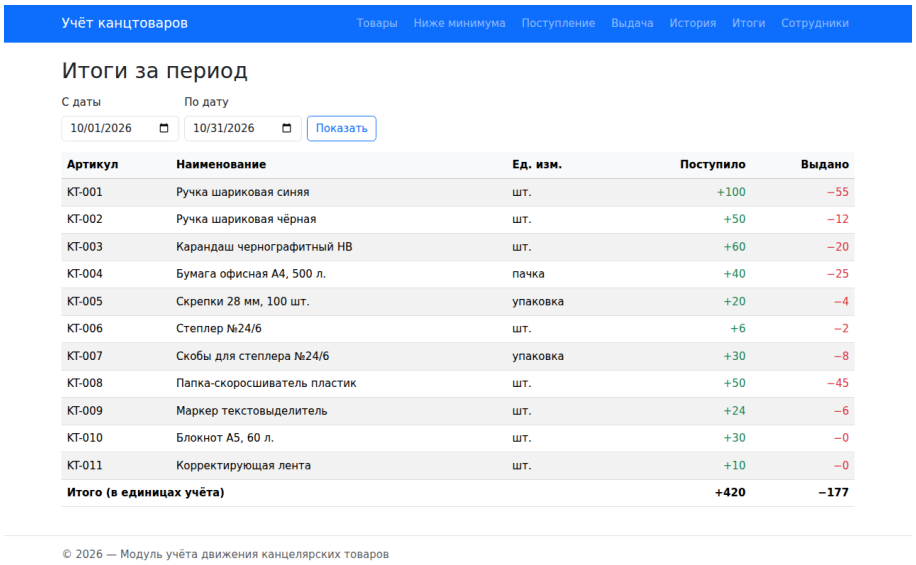


Рисунок 8 – Итоги поступлений и выдач за октябрь 2026 года

3.5 Хранение данных и развёртывание

База SQLite создаётся миграцией InitialCreate. При запуске приложение вызывает метод Database.Migrate(), который создаёт файл stationery.db и таблицы, если их ещё нет, а затем DbSeeder добавляет учебные данные: 12 товаров, 8 сотрудников и 27 операций за октябрь – ноябрь 2026 года. Если товары в базе уже есть, учебные данные повторно не добавляются, поэтому изменения пользователя сохраняются между запусками. Чтобы вернуть учебную базу в исходное состояние, достаточно остановить приложение и удалить файл stationery.db.

Для развёртывания нужен только .NET SDK 10: пакеты NuGet скачиваются автоматически при первой сборке. Порядок запуска описан в файле README.md проекта. Исходный код хранится в репозитории Git; каждый этап работы зафиксирован отдельными коммитами. После каждой отправки изменений на GitHub автоматически выполняются сборка и все тесты (GitHub Actions).

3.6 Тестирование программного модуля

Подход к тестированию. Автоматическими тестами проверяется предметная логика, которую можно проверить отдельно от интерфейса: расчёт остатка, главное правило выдачи, контроль минимума, проверки ввода, поиск, фильтры, итоги и сохранение данных. Тесты вызывают методы сервисов напрямую и используют базу SQLite в оперативной памяти: для каждого теста создаётся новая пустая база, поэтому тесты не зависят друг от друга. Работа интерфейса, сохранение после перезапуска приложения и развёртывание проверены вручную через браузер и HTTP-запросы.

Каждый тест построен по схеме «подготовка – действие – проверка». В листинге 6 приведён тест граничного случая главного правила: при остатке 10 шт. выдача 11 шт. должна быть отклонена, а остаток не должен измениться.

Листинг 6 – Тест граничного случая «остаток + 1»

(StationeryAccounting.Tests/IssueTests.cs)

```
[Fact]
public void ВыдатьТовар_ОстатокПлюсОдин_Отклоняется()
{
    // Подготовка (в конструкторе класса): ручка, поступило 10 шт., минимум 3
    // Действие:
    var result = Issue(11);

    // Проверка: выдача отклонена, остаток прежний
    Assert.False(result.Success);
    Assert.Equal(10, Balance());
}
```

Тестовый проект StationeryAccounting.Tests содержит 36 тестовых методов; с учётом параметризованных тестов (Theory) выполняется 48 проверок. Состав тестов приведён в таблице 5, полный код тестов – в проекте StationeryAccounting.Tests, а тесты расчёта остатка – в приложении А.

Таблица 5 – Состав автоматических тестов

Тестовый класс	**Функции**	**Проверок**	**Что проверяется**	**Результат**
StockCalculatorTests	Ф4, Ф6	7	формула остатка; признак «Требует пополнения» при остатке меньше, равном и больше минимума	пройдены
IssueTests	Ф3, Ф6	13	главное правило; выдача ровно всего остатка и «остаток + 1»; пустой получатель, количество 0 и −5, дата в будущем, архивный товар, неизвестные Id; предупреждение о минимуме	пройдены
ReceiptTests	Ф2	6	поступление увеличивает остаток; обязательность поставщика или документа; архивный товар; количество 0; неизвестный товар	пройдены
ProductServiceTests	Ф1, Ф4, Ф6	13	добавление с нормализацией, повтор артикула, пустое наименование, минимум −1 и 0, неизвестный Id, повторное архивирование, поиск, фильтры	пройдены
HistoryAndTotalsTests	Ф5, Ф7	6	история без фильтра, по товару и типу, по сотруднику, по периоду с включёнными границами; итоги за период и за период без операций	пройдены
EmployeeAndPersistenceTests	Ф8, сохранение	3	пустое ФИО, неизвестный Id; запись в одном DbContext и чтение в новом	пройдены

Тесты запущены командой dotnet test на сервере GitHub Actions (Ubuntu, .NET 10). Результат: всего 48 проверок, пройдено 48, не пройдено 0 (рисунок 9). Тесты расчёта остатка приведены в приложении А.

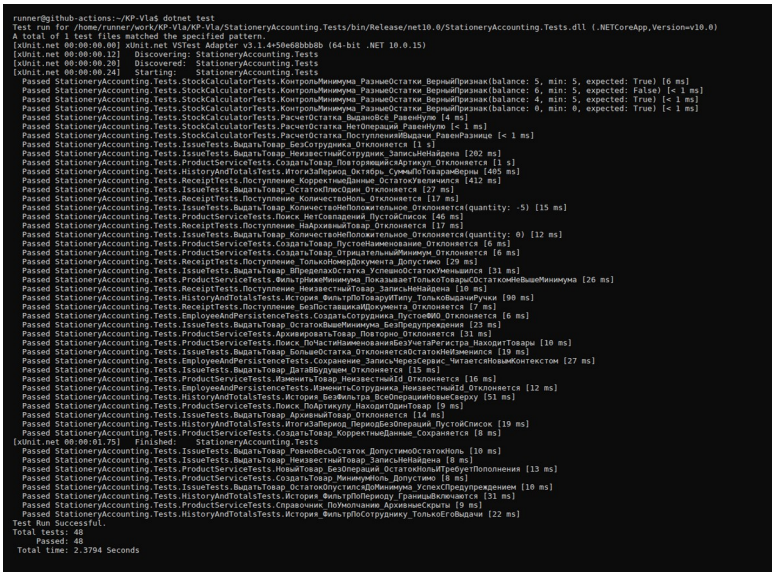


Рисунок 9 – Результаты выполнения автоматических тестов

Ручные проверки. Сценарии, связанные с интерфейсом и перезапуском приложения, проверены в браузере. Проверки выполнялись автоматизированным браузером (Google Chrome под управлением Playwright) по сценарию, записанному в файле tools/screenshots.py: сценарий выполнял действия пользователя, читал со страницы фактически появившееся сообщение и сохранял снимок экрана. Фактические результаты – это тексты, прочитанные со страниц, а не ожидаемые значения. Всего выполнено 25 ручных проверок, все дали ожидаемый результат; например, выдача 100 шт. при остатке 38 отклонена с сообщением «Недостаточно товара», а после перезапуска приложения добавленный товар и остатки сохранились. Полная таблица из 25 проверок оформлена отдельным документом «Таблица проверок».

Развёртывание из архива проверено так: архив проекта, созданный командой git archive, распакован в новую папку, после чего выполнены шаги инструкции запуска из README.md – сборка, автоматическое создание базы, открытие страниц. На сервере GitHub Actions та же последовательность выполняется при каждом запуске проверок на чистой машине без заранее созданной базы.

Найденные ошибки и их исправления приведены в таблице 6.

Таблица 6 – Найденные ошибки и исправления

№	Ошибка	Причина	Исправление	Повторная проверка
1	В предупреждении выводилось «3 упаковка»	единица измерения подставлялась в полной форме без согласования с числом	добавлены сокращения единиц (Units.Short): «3 уп.»	проверка 14 повторена: «3 уп.»
2	Ошибка проверки данных в API выводилась с английским заголовком «One or more validation errors occurred»	использовался стандартный ответ ASP.NET Core	задан собственный ответ { error } с текстами на русском	проверка 13 повторена: ответ на русском
3	Первый запуск не создавал таблицы	отсутствовала миграция при вызове Database.Migrate()	создана миграция InitialCreate	база создаётся при каждом чистом запуске

Оставшиеся ограничения: нет входа в систему и ролей (не предусмотрено заданием); стили Bootstrap подключаются из сети.

Выводы по тестированию. Все 48 автоматических проверок пройдены, все 25 ручных проверок дали ожидаемый результат. Главное правило и все граничные случаи – выдача ровно всего остатка, «остаток + 1», остаток, равный минимуму, новый товар без операций – подтверждены тестами. Найденные в ходе проверок ошибки исправлены, проверки повторены.

Выводы по разделу 3

Реализованы все восемь обязательных функций: справочник товаров с поиском и архивом, поступление и выдача, автоматический расчёт остатка, история движения с фильтрами, контроль минимума, итоги за период и справочник сотрудников. Предметная логика сосредоточена в сервисах и проверена 48 автоматическими тестами; интерфейс и сохранение данных после перезапуска проверены вручную. Модуль разворачивается одной командой, база создаётся автоматически.

ЗАКЛЮЧЕНИЕ

В курсовом проекте разработан модуль учёта движения канцелярских товаров. Цель работы достигнута, что подтверждается решением каждой задачи.

Предметная область проанализирована в разделе 1: выявлены проблемы ручного учёта, рассмотрены аналоги, сформулированы восемь функциональных требований и ограничения, выбраны средства разработки – C#, .NET 10, ASP.NET Core MVC, EF Core с SQLite, xUnit и Git.

Проектирование выполнено в разделе 2: модель данных из трёх сущностей, формула остатка «сумма поступлений минус сумма выдач» без хранения остатка в базе, трёхслойная архитектура и алгоритм выдачи с защитой от одновременных операций.

Реализация описана в подразделах 3.1–3.5: все функции задания работают, неверные данные отклоняются с понятными сообщениями, неизвестная запись даёт страницу «Запись не найдена».

Тестирование выполнено в подразделе 3.6: 48 автоматических и 25 ручных проверок пройдены, найденные ошибки исправлены. Подготовлены инструкция запуска и материалы к защите. Ключевой код приведён в приложениях А и Б.

Перспективы развития: роли и вход в систему, учёт стоимости, заказ поставщику по товарам ниже минимума, выгрузка в Excel.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Российская Федерация. Законы. О бухгалтерском учете : Федеральный закон № 402-ФЗ : [принят Государственной думой 22 ноября 2011 года : одобрен Советом Федерации 29 ноября 2011 года]. – Текст : электронный // КонсультантПлюс. – URL: https://www.consultant.ru/document/cons_doc_LAW_122855/ (дата обращения: 11.11.2026).

2. Об утверждении Федерального стандарта бухгалтерского учета ФСБУ 5/2019 «Запасы» : Приказ Министерства финансов Российской Федерации от 15 ноября 2019 г. № 180н. – Текст : электронный // КонсультантПлюс. – URL: https://www.consultant.ru/document/cons_doc_LAW_339442/ (дата обращения: 11.11.2026).

3. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем. Обозначения условные и правила выполнения : издание официальное : утвержден и введен в действие Постановлением Государственного комитета СССР по управлению качеством продукции и стандартам от 26.12.90 № 3294 : дата введения 1992-01-01. – Москва : Стандартинформ, 2010. – 22 с.

4. Лок, Э. ASP.NET Core в действии / Э. Лок ; перевод с английского Д. А. Беликова. – Москва : ДМК Пресс, 2021. – 906 с. – ISBN 978-5-97060-550-9. [проверить]

5. Прайс, М. C# 10 и .NET 6. Современная кросс-платформенная разработка / М. Прайс. – Санкт-Петербург : Питер, 2023. – 848 с. – ISBN 978-5-4461-2249-0. [проверить]

6. Смит, Д. Entity Framework Core в действии / Д. Смит ; перевод с английского Д. А. Беликова. – Москва : ДМК Пресс, 2022. – 690 с. – ISBN 978-5-93700-114-6. [проверить]

7. Хориков, В. Принципы юнит-тестирования / В. Хориков. – Санкт-Петербург : Питер, 2021. – 320 с. – ISBN 978-5-4461-1683-3.

8. Чакон, С. Git для профессионального программиста / С. Чакон, Б. Штрауб. – Санкт-Петербург : Питер, 2016. – 496 с. – ISBN 978-5-496-01763-3.

9. Overview of ASP.NET Core MVC. – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/aspnet/core/mvc/overview (дата обращения: 11.11.2026).

10. Entity Framework Core. – Текст : электронный // Microsoft Learn. – URL: https://learn.microsoft.com/ef/core/ (дата обращения: 11.11.2026).

Приложение А

Листинг тестов расчёта остатка

Файл StationeryAccounting.Tests/StockCalculatorTests.cs

```
using StationeryAccounting.Models;
using StationeryAccounting.Services;

namespace StationeryAccounting.Tests;

public class StockCalculatorTests
{
    private static StockOperation Op(OperationType type, int quantity) =>
        new() { Type = type, Quantity = quantity };

    [Fact]
    public void РасчетОстатка_ПоступленияИВыдачи_РавенРазнице()
    {
        // 100 + 20 поступило, 30 + 25 выдано → 65
        var ops = new[]
        {
            Op(OperationType.Receipt, 100), Op(OperationType.Receipt, 20),
            Op(OperationType.Issue, 30), Op(OperationType.Issue, 25)
        };

        Assert.Equal(65, StockCalculator.CalculateBalance(ops));
    }

    [Fact]
    public void РасчетОстатка_НетОпераций_РавенНулю()
    {
        Assert.Equal(0,
StockCalculator.CalculateBalance(Array.Empty<StockOperation>()));
    }

    [Fact]
    public void РасчетОстатка_ВыданоВсё_РавенНулю()
    {
        var ops = new[] { Op(OperationType.Receipt, 10), Op(OperationType.Issue,
10) };

        Assert.Equal(0, StockCalculator.CalculateBalance(ops));
    }

    [Theory]
    [InlineData(4, 5, true)]    // меньше минимума
    [InlineData(5, 5, true)]    // ровно минимум — уже «Требует пополнения»
    [InlineData(6, 5, false)]   // больше минимума
    [InlineData(0, 0, true)]    // минимум 0, остатка нет
    public void КонтрольМинимума_РазныеОстатки_ВерныйПризнак(int balance, int
min, bool expected)
    {
        Assert.Equal(expected, StockCalculator.NeedsRestock(balance, min));
    }
}
```

Приложение Б

Листинг класса результата операции

Файл Services/OperationResult.cs

```
namespace StationeryAccounting.Services;

public class OperationResult
{
    public bool Success { get; private set; }

    public string? Error { get; private set; }

    public string? Warning { get; private set; }

    public bool IsNotFound { get; private set; }

    public static OperationResult Ok(string? warning = null)
    {
        return new OperationResult { Success = true, Warning = warning };
    }

    public static OperationResult Fail(string error)
    {
        return new OperationResult { Success = false, Error = error };
    }

    public static OperationResult NotFound(string error)
    {
        return new OperationResult { Success = false, Error = error, IsNotFound
= true };
    }
}
```